

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name: Student ID: Zhanaliyeva Meruyert , 230103316

Hardware / CPU: Date & Section: Apple M2

1. Table 1

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	0.9880	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	0.5089	1.94x	97.1%
Challenge 1: Monte Carlo	4 Threads	120,000,000	0.2668	3.70x	92.6%
Challenge 1: Monte Carlo	Max Threads (8)	120,000,000	0.2336	4.23x	52.9%
Challenge 2: Mandelbrot (Rows)	Max Threads (8)	2500x2500	1.561	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads (8)	2500x2500	1.509	N/A	N/A
Challenge 3: Heat Stencil	Max Threads (8)	1500x1500x300	0.149	N/A	N/A

(float64)					
Challenge 3: Heat Stencil (float32)	Max Threads (8)	1500x1500x 300	0.076	N/A	N/A

2. Technical Answers to Questions

1. Deliverables & Technical Questions for Challenge 1

A. Baseline & Max Runtime

T1 (1 thread): **0.9880 s**

Tmax (8 threads): **0.2336 s**

B. Why warm-up is required : It triggers JIT compilation beforehand. Without it, the first timing includes compilation overhead instead of just execution speed.

C. Parallel efficiency & degradation

No, efficiency dropped from **100% to 52.9%**.

This happens because:

1. **P/E Cores:** Scaling to 8 threads forces work onto slower Efficiency cores (Apple Silicon).

Overhead & Memory: Thread synchronization costs and memory bus competition increase with more threads.

D. Why **inside_circle += 1** doesn't corrupt data Numba automatically creates an **OpenMP reduction** construct. Each thread keeps its own local counter during the loop, and Numba sums them all up safely at the end to prevent race conditions.

2. Deliverables & Technical Questions for Challenge 2

A. Comparison of t_rows vs t_cols

Results: t_rows = **1.561 s**, t_cols = **1.509 s**

Analysis: Both axis decompositions executed at comparable speeds with a minor advantage for column-parallel rendering. This occurs because Numba's dynamic thread scheduling effectively balances task distribution across available cores regardless of the primary loop axis.

B. C-Contiguous Memory Layout & CPU Cache Lines

NumPy arrays use a **C-contiguous (row-major)** layout in memory, meaning adjacent elements in a row are stored right next to each other in RAM.

`img[r, c]` (Row-major access): Threads write to contiguous memory addresses, loading entire 64-byte CPU cache lines efficiently with high cache hit rates.

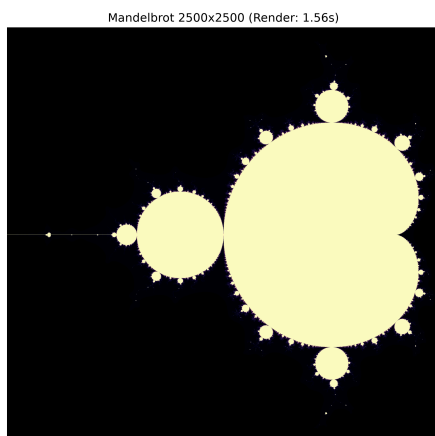
`img[c, r]` (Column-major access): Threads write with large memory strides, causing frequent cache misses and memory bus bandwidth waste.

C. Load Imbalance & Dynamic Scheduling

Why one thread idles while another struggles: Points outside the Mandelbrot set escape in 2–3 iterations, whereas points in the center require all 1000 iterations. Thread 0 (top 20% of rows) processes mostly background space and finishes instantly, while Thread 2 (center 20%) handles complex set boundaries and gets overloaded.

How OpenMP Dynamic Scheduling fixes it: Instead of splitting rows evenly up front (static scheduling), dynamic scheduling assigns small chunks of rows on-demand. Whenever a thread finishes its workload, it immediately requests another chunk, keeping all CPU cores 100% utilized

D. Exported Artifact



3. Deliverables & Technical Questions for Challenge 3

A. Benchmark Metrics (Table 1)

Runtime (np.float64): 0.149 s

Throughput (np.float64): 4522.71 Megacells/sec

B. Array Precision Change (np.float32 vs np.float64)

Runtime Decrease Factor: Runtime dropped from **0.149 s** down to **0.076 s**, decreasing by a factor of **1.96x** (nearly half the execution time).

PDF

Why halving memory footprint improves throughput: Stencil computations perform very few arithmetic operations per memory access, making them memory-bandwidth bound rather than compute-bound. Halving the precision from 64-bit to 32-bit floats halves the byte volume transferred over the RAM bus, effectively doubling memory throughput.

C. Memory Roofline Model

According to the Roofline Model, overall throughput is capped by whichever hardware bottleneck is reached first: CPU FLOPS or RAM Bandwidth.

Once the memory bus is completely saturated transferring grid data, the CPU cores spend most of their clock cycles stalled waiting for data from main memory. Adding twice as many CPU cores increases arithmetic capability, but it cannot overcome the physical RAM bandwidth wall.